Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Специальность «Программная инженерия»

Кафедра программного обеспечения информационных технологий

Учебная дисциплина «Компьютерные системы и сети»

ОТЧЕТ

по лабораторной работе №3
«Чат»

Подготовила: Студент гр. 410902
Кухто В.С.

Проверила: Болтак С.В.

Минск 2026

Условие задания: Необходимо разработать программу (консольную или графическую) для обмена текстовыми сообщениями, работающую в локальной сети в одноранговом режиме.

Каждый участник обмена сообщениями (узел) идентифицируется IP-адресом и произвольным именем, которое задается пользователем (через параметр командной строки, конфигурационный файл или любым другим способом). Уникальность имен не требуется.

Каждый узел с помощью UDP формирует список активных узлов (IP-адреса и имена):

после запуска узел отправляет широковещательный пакет, содержащий свое имя, для уведомления других узлов в сети о своем подключении к сети;

другие узлы, получившие такой пакет, устанавливают с отправителем TCP-соединение для обмена сообщениями и передают по нему свое имя для идентификации в чате.

В любой момент к чату может присоединиться новый клиент.

Обмен сообщениями ведется с помощью TCP в логически общем пространстве: каждый узел поддерживает по одному TCP соединению с каждым другим узлом и отправляет свои сообщения всем узлам в сети. Отключение узла должно корректно обрабатываться другими узлами. Пользовательский интерфейс программы должен позволять вводить с клавиатуры и отправлять сообщения, а также просматривать историю событий с момента последнего запуска программы.

История должна включать следующие сообщения в хронологическом порядке с отметками времени:

входящие сообщения от других узлов (с указанием имени и IP-адреса отправителя);

собственные отправленные сообщения; обнаружение нового узла; отключение работающего узла.

Для обмена сообщениями рекомендуется разработать свой собственный формат сообщения, позволяющий передавать сообщения разных типов и упрощающих передачу сообщений в потоковом режиме, используемом в TCP.

Например:

смещение в байтах	тип сообщения
0	тип сообщения (1 - сообщение, 2 - передача имени, 3 - пользователь подключен, 4 - пользователь отключен и т.п.)
1	длина сообщения в байтах
далее n байт	содержание сообщения

Способы тестирования чата в многопользовательском режиме:

Docker (для консольных программ).

Виртуальные машины VirtualBox (для графических программ).

Bind на разные адреса loopback интерфейса: 127.0.0.1, 127.0.0.2 и т. д.

При создании сокетов на прослушивание входящих широковещательных UDP-пакетов и TCP-соединений нужно указывать IP-адрес, переданный при запуске программы (параметр командной строки, конфигурационный файл и т.п.). Далее запустить несколько экземпляров программы, передавая разные IP-адреса: 127.0.0.1, 127.0.0.2 и т.д. Все эти адреса будут ассоциированы с вашим компьютером, однако логически программы будут разными независимыми клиентами.

Дополнительное задание

Реализовать передачу имеющейся истории событий узлу при подключении.

После обнаружения узлов новый узел запрашивает у любого имеющегося историю событий. По TCP-соединению, установленному между этими узлами, происходит передача полной истории событий, известной узлу. После приема история должна быть отображена соответствующим образом в пользовательском интерфейсе.

Выполнение работы:

Написание было выполнено на языке Python. Разработка P2P-чата происходила в среде Anaconda, в приложении Spyder. Запустить консольную программу можно, скачав файл chat.py, bat-файл mychat.py в одну папку, затем перейти в терминал в этой папке и ввести mychat 127.0.0.1 (также 127.0.0.2, 127.0.0.3). Программа также запускается в терминале Spyder, VS Code и т.д.

Запуск написанной программы проводился на одном компьютере с использованием разных IP-адресов loopback-интерфейса (127.0.0.1, 127.0.0.2, 127.0.0.3).

На рисунке можно увидеть, что при запуске bat-файла пользователь сам может ввести IP-адрес, имя узла, номер UDP-порта, TCP-порта. Затем пользователь может просмотреть историю, доступные узлы, введя определенные команды:

```
CHAT

Enter IP address : 127.0.0.1
Enter your name: 1
Enter UDP port: 8888
Enter TCP port: 8889

Available commands:
  /history - show message history
  /nodes   - show connected nodes
  /exit    - exit the program

Starting with: IP=127.0.0.1, NAME=1, UDP=8888, TCP=8889

Node started: 1 (127.0.0.1:8889)
```

Рисунок 1

При запуске узла с параметрами IP=127.0.0.1, имя=1, UDP-порт=8888, TCP-порт=8889 отправлено UDP-широковещательное сообщение.

Для захвата трафика потребовалась установка Npcap с опцией поддержки loopback-трафика. В Wireshark (фильтр udp) можно увидеть отправку пакета с источником 127.0.0.1 на широковещательный адрес 255.255.255.255 (рисунок 2):

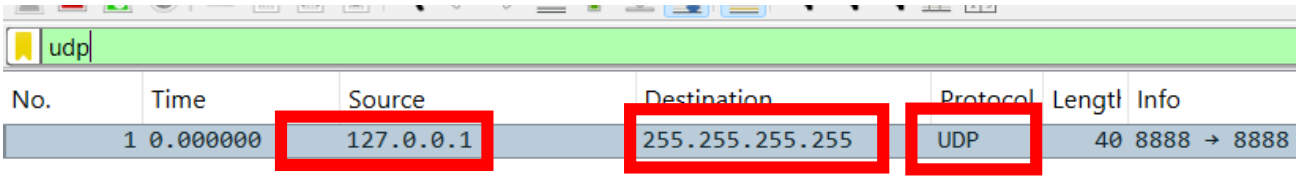

Рисунок 2

Также можно увидеть данные, которые были переданы (рисунок 3):

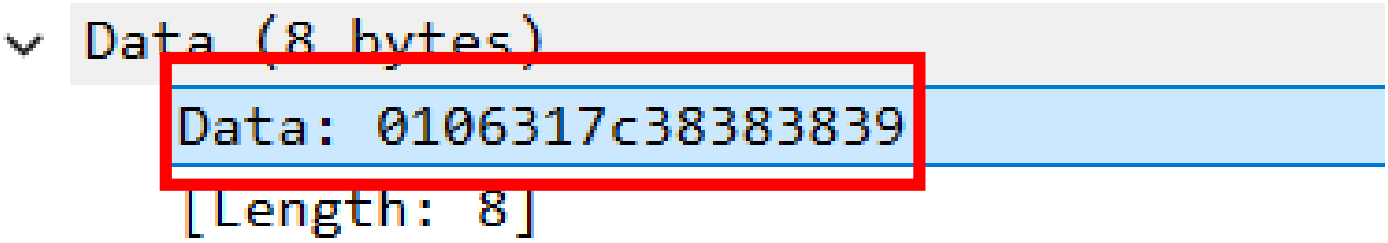

Рисунок 3

В данном представлении мало понятно, что они означают, но нажав правой кнопкой мыши по ним, выбрав пункт «Копировать как UTF-8» данные сохраняются в буфер обмена как - 1|8889, что означает имя узла и номер tcp-порта.

Для обмена данными между узлами был разработан собственный бинарный протокол. Каждое сообщение имеет следующую структуру:

- байт 0: тип сообщения (1 - HELLO, 2 - NAME, 3 - MSG, 4 - GET_HISTORY, 5 - HISTORY, 6 - DISCONNECT)
- байт 1: длина данных в байтах (0-255)
- байты 2...+длина: содержимое сообщения в кодировке UTF-8

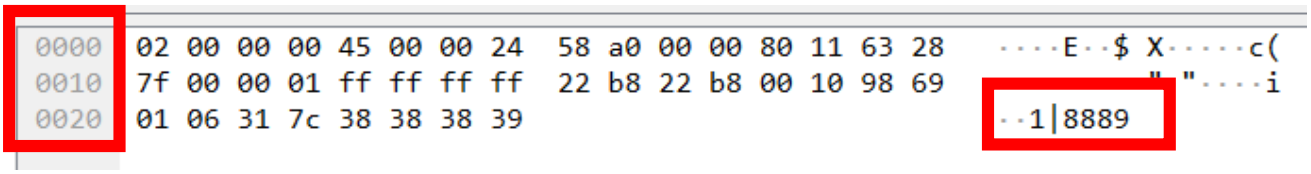

Рисунок 4

В таблице 1 можно увидеть разработанный протокол:

Тип сообщения	Значение константы	Формат данных
HELLO	1	имя_узла|порт
NAME	2	имя_узла|порт

MSG	3	ID_сообщения|отправитель|текст
GET_HISTORY	4	-
HISTORY	5	запись1;;;запись2;;;;….
DISCONNECT	6	-

Таблица 1

На рисунке 5.1 можно увидеть, что происходит, если второй узел пытается подключиться на том же самом ip-адресе и tcp-порту, что и существующий узел. Программа просит ввести корректные значения:

```
CHAT

Enter IP address : 127.0.0.1
Enter your name: 2
Enter UDP port: 8888
Enter TCP port: 8889

Available commands:
  /history - show message history
  /nodes   - show connected nodes
  /exit    - exit the program

Starting with: IP=127.0.0.1, NAME=2, UDP=8888, TCP=8889

Error: TCP port 8889 is already in use

Invalid input detected! Please try again.
```

Рисунок 5.1

Теперь протестируем запуск на разных ip-адресах, но на одном порту. Параллельно с первым узлом запускаю второй узел (рисунок 6).

Рисунок 6

На рисунке 7 можно увидеть TCP-рукопожатие. Узлы (uz1, uz2) обмениваются именами для идентификации в чате. SYN - первый этап трехэтапного рукопожатия TCP, здесь Sequence number случайное число (0). SYN, ACK - второй этап, подтверждение получения SYN, Acknowledgment number =1 (0+1), Sequence number=0. ACK - третий этап, соединение установлено, Acknowledgment number =1 (0+1). Соединение установлено:

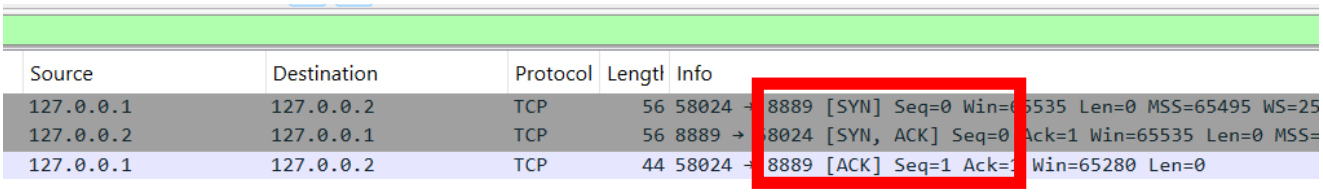

Рисунок 7

На рисунке 8 можно увидеть, как происходит процесс отправки сообщений от первого узла ко второму и наоборот:

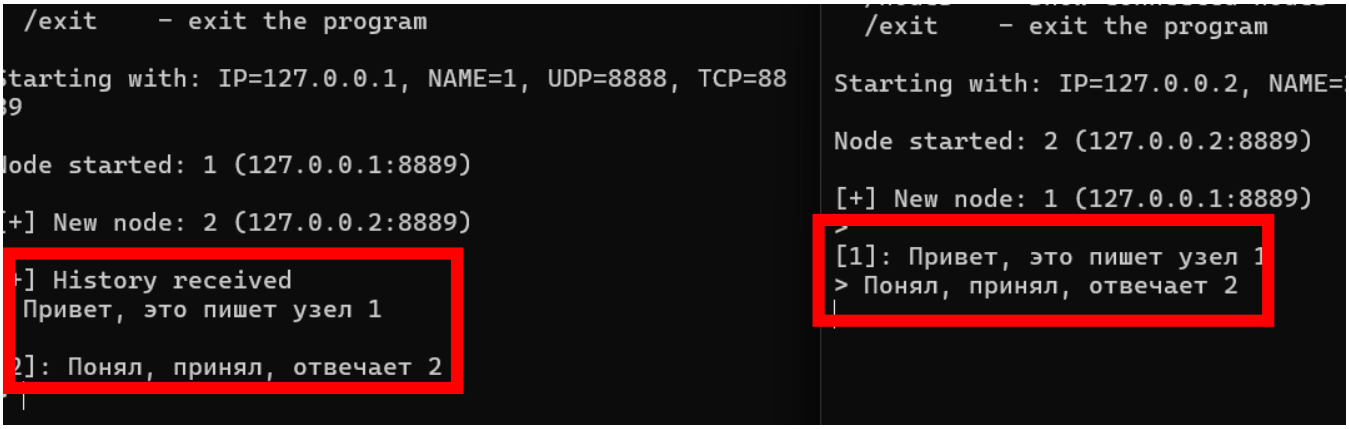

Рисунок 8

В WireShark при этом можно увидеть отправленные сообщения (рисунок 9):

127.0.0.1	127.0.0.2	TCP	125	52198 → 8889 [PSH, ACK] Seq=11 Ack=92 Win=65280 Len=81
127.0.0.2	127.0.0.1	TCP	44	8889 → 52198 [ACK] Seq=92 Ack=92 Win=65280 Len=0
127.0.0.2	127.0.0.1	TCP	128	8889 → 52198 [PSH, ACK] Seq=92 Ack=92 Win=65280 Len=84
127.0.0.1	127.0.0.2	TCP	44	52198 → 8889 [ACK] Seq=92 Ack=176 Win=65280 Len=0

Рисунок 9

На рисунке 9 видно, кто отправил эти сообщения и кому они адресуются (сначала отправлял 1 узел, затем 2). Также можно узнать, что за сообщение было передано (способ был описан ранее, правая кнопка мыши - копировать как текст UTF-8). Переданные данные от первого узла 1: R127.0.0.1_8889_1_1773643325.1926959|1|Привет, это пишет узел 1 от второго T127.0.0.2_8889_1_1773643336.39775322|Понял, принял, отвечает 2. Первые символы 'R' и 'T' в данных - это отображение типа сообщения (3 - MSG), так как Wireshark интерпретирует бинарные данные как текст.

При этом можно увидеть, что сообщения, а также подключения новых узлов сохраняются в истории (рисунок 10):

```
> /history

--- HISTORY ---
[00:34:57] SYSTEM: Node started
[00:35:10] SYSTEM: New node: 2 (127.0.0.2:8889)
[00:35:10] SYSTEM: Node started
[00:35:10] SYSTEM: New node: 1 (127.0.0.1:8889)
[00:36:57] 1: Привет, это пишет узел 1
[00:37:08] 2: Понял, принял, отвечает 2
```

Рисунок 10

На рисунке 11 можно увидеть, что происходит при подключении 3 узла:

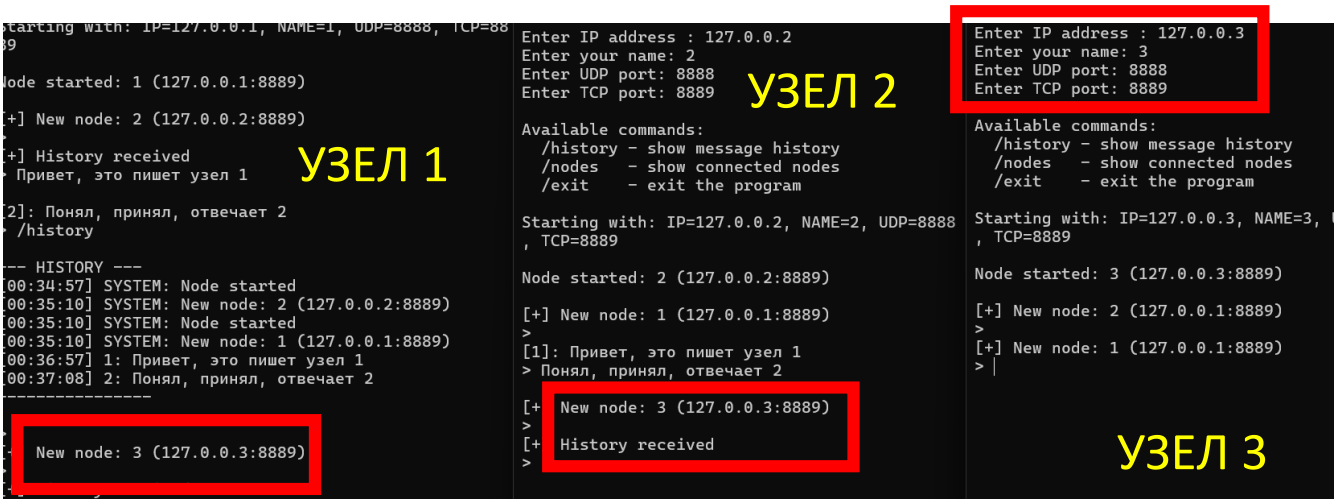

Рисунок 11

При этом в WireShark можно увидеть, как была передана история при подключении (случай для 3 узлов) (рисунок 12):

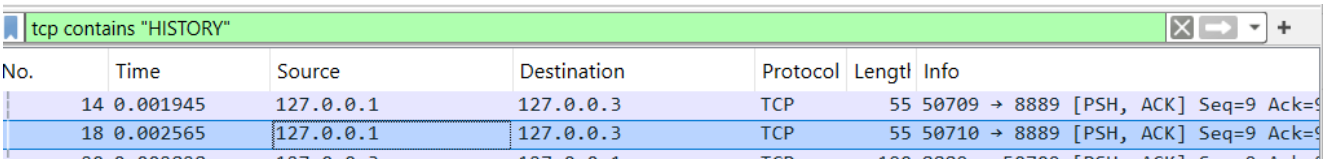

Рисунок 12

Опять преобразовав данные в формат UTF-8 можно увидеть [09:45:58] SYSTEM: Node started on port 8889;;;[09:45:58] SYSTEM: New node: uz2 (127.0.0.1:8889);;;[09:45:58] SYSTEM: New node: uz1 (127.0.0.1:8889)).

На рисунке 13 можно увидеть просмотр первым узлом полученной истории:

```
--- HISTORY ---
[09:28:12] SYSTEM: Node started on port 8889
[09:40:10] SYSTEM: New node: uz2 (127.0.0.2:8889)
[09:40:10] SYSTEM: Node started on port 8889
[09:40:10] SYSTEM: New node: uz1 (127.0.0.1:8889)
[09:42:05] uz1: Привет, это пишет узел 1
[09:42:16] uz2: Понял, принял, отвечает 2
[09:45:58] SYSTEM: New node: uz3 (127.0.0.3:8889)
[09:45:58] SYSTEM: Node started on port 8889
[09:45:58] SYSTEM: New node: uz2 (127.0.0.1:8889)
[09:45:58] SYSTEM: New node: uz1 (127.0.0.1:8889)
----------------
```

УЗЕЛ 1 запущен

УЗЕЛ 1 заметил узел 2

История от узла 2

УЗЕЛ 1 заметил узел 3

История от узла 3

Рисунок 13

На рисунке 14 можно увидеть, что отправка третьим узлом сообщения тоже происходит корректно.

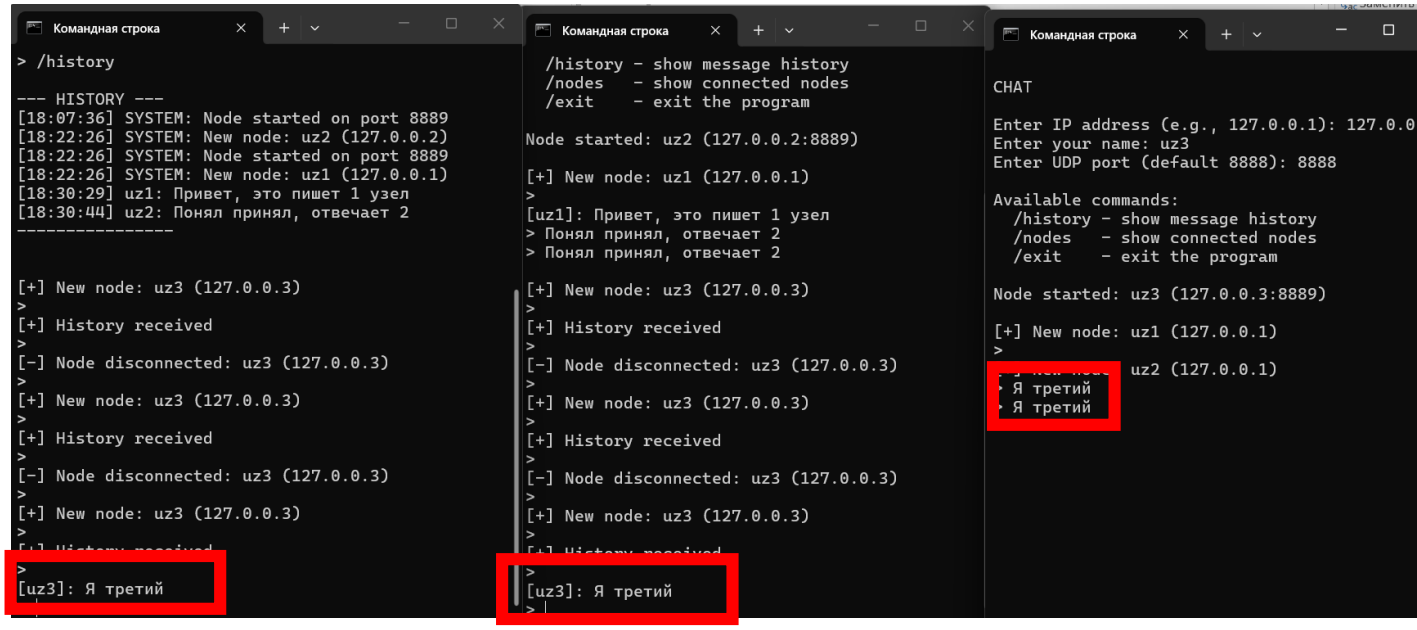

Рисунок 14

На рисунке 15 можно увидеть, как работает функция просмотра активных узлов (просмотр у первого узла), видны адреса и имена узлов:

```
> /nodes

--- NODES ---
  uz2 (127.0.0.2)
  uz3 (127.0.0.3)
-------------
```

Рисунок 15

На рисунке 16 можно увидеть, как происходит отключение узла (здесь отключился первый):

```
/exit

> Program terminated
Press any key to continue . . .
```

```
[+] History received
>
[uz3]: Я третий
>
[-] Node disconnected: uz1 (127.0.0.1)
```

Рисунок 16

Просмотрев историю для второго узла можно увидеть соответствующее сообщение об отключении первого (рисунок 17):

```
--- HISTORY ---
[23:01:15] SYSTEM: Node started on port 8889
[23:01:15] SYSTEM: New node: uz1 (127.0.0.1:8889)
[23:01:31] SYSTEM: New node: uz3 (127.0.0.3:8889)
[23:01:31] SYSTEM: Node disconnected: uz3 (127.0.0.3:8889)
[23:02:47] SYSTEM: Node disconnected: uz1 (127.0.0.1:8889)
----------------
```

Рисунок 17

Просмотрев вторым узлом список активных, теперь там будет только третий узел (рисунок 18):

```
/nodes

--- NODES ---
  3 (127.0.0.3:8889)
--------------
```

Рисунок 18

На рисунке 19 можно увидеть, что наблюдается в WireShark при отключении 1 узла:

127.0.0.1	127.0.0.2	TCP	44	52198 → 889 [ACK] Seq=3 Ack=46 Win=255 Len=0
127.0.0.3	127.0.0.1	TCP	44	8889 → 60874 [FIN, ACK] Seq=45 Ack=47 Win=255 Len=0
127.0.0.1	127.0.0.2	TCP	44	52198 → 889 [FIN, ACK] Seq=3 Ack=46 Win=255 Len=0
127.0.0.1	127.0.0.3	TCP	44	60874 → 889 [ACK] Seq=47 Ack=46 Win=255 Len=0
127.0.0.2	127.0.0.1	TCP	44	8889 → 52198 [ACK] Seq=46 Ack=4 Win=255 Len=0
127.0.0.1	127.0.0.3	TCP	44	60874 → 889 [FIN, ACK] Seq=47 Ack=46 Win=255 Len=0
127.0.0.3	127.0.0.1	TCP	44	8889 → 60874 [ACK] Seq=46 Ack=48 Win=255 Len=0

Рисунок 19

Перед закрытием соединения узел отправляет специальное сообщение DISCONNECT (тип 6), после чего инициируется штатное завершение TCP-соединения с обменом FIN-пакетами. Узел 2 отправляет FIN, ACK узлу 1 (хочет закрыть соединение). Узел 1 подтверждает (ACK) получение FIN от узла 2. Узел 3 отправляет FIN, ACK узлу. Узел 1 подтверждает (ACK) получение FIN от узла 3. Узел 1 отправляет FIN, ACK узлу 2 (закрывает свою сторону). Узел 2 подтверждает (ACK) получение FIN от узла 1. Узел 1 отправляет FIN, ACK узлу 3. Узел 3 подтверждает (ACK) получение FIN от узла 1. Соединение закрыто.